



Introduction to OpenShift AI

A Generative AI & RAG Workshop for Students

Christian Zaccaria

**| Table of contents we'll go through.
OpenShift AI with a focus on RAG**

Introduction to OpenShift AI

| What is OpenShift AI?

An AI Platform for Everyone

A platform for building, running, and managing AI applications. Bringing your solution from experimentation to production.

- ✓ **One place for the full AI workflow:** train models, serve them, and monitor them.
- ✓ **Consistent Experience:** Works the same on your laptop or a cloud cluster.
- ✓ **Ready for GenAI:** High-performance tools for LLMs and RAG.
- ✓ **Runs anywhere:** cloud, on-premise, or at the edge.
- ✓ **Open Source Roots:** uses tools you may already know; PyTorch, Kubeflow, MLflow
- ✓ **Ready for GenAI:** High-performance AI tools for LLMs, RAG, and more.



| The Analogy and problems it solves for enterprises

The analogy: think of it like a kitchen

Imagine you're a chef. OpenShift AI is the fully equipped and professional-grade kitchen:

- ✓ **It provides everything you need:** Ovens, Fridges, Pans, Utensils, Ingredients, and the kitchen itself; while still allowing you to bring your own models, tools, and data if you want to.
- ✓ But, you're the one cooking the dish. You bring the creativity, the expertise, and the recipe. OpenShift AI just makes sure you have everything you need to go from an idea to a production grade solution.

Problems faced by enterprises

Teams were stitching together dozens of disconnected tools just to bring an AI solution from a notebook into production. Slow, fragile, and hard to scale. Managing infrastructure, security, and compliance on top of that made AI feel more like an ops burden than an innovation driver.

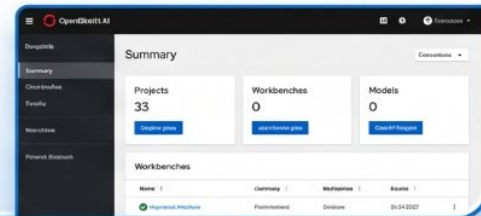


In OpenShift AI: we take care of the infrastructure, components, integration, security, and management of these components, while you solely focus on creating your AI solution



OpenShift AI Dashboard

Platform UI for project management, workbenches, models, and settings.



Deploy & Serve

OpenShift AI Platform



MODEL DEVELOPMENT

WORKBENCHES



Unified distributed training across multiple nodes and GPUs.

DISTRIBUTED TRAINING & WORKLOADS



Training Operator (Kubeflow Trainer)
TrainJob API and ClusterTrainingRuntime templates (PyTorch, JAX, etc.).
Adheres to DataScienceCluster.

Workflow

AI PIPELINES



AI PIPELINES

Workflow automation (data prep, training, eval). Repeatable.

Deploy & Serve



MODEL MANAGEMENT



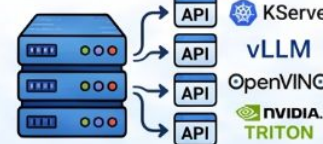
MODEL REGISTRY

Catalog to version, track, and share models.

ML Features



MODEL DEPLOYMENT



MODEL SERVING

Deploy models as scalable API endpoints.



TRUSTYAI

Explainability, bias detection, and drift monitoring.



LLAMA STACK (OGX)

Unified APIs for RAG, safety, tool access (MCP), memory.

Experiment & observe GenAI models (Llama, mistral), RAG, and safety settings in a user-friendly UI.



RED HAT OPENSIFT CONTAINER PLATFORM



INFRASTRUCTURE: HYBRID CLOUD, ON-PREMISE, EDGE



| OpenShift AI components we'll use during the workshop



OpenShift AI Dashboard

The central hub to manage your Data Science projects and infrastructure resources.



Workbenches

Isolated coding environments with pre-installed AI libraries and tools.



GenAI Playground

Interactive playground to quickly test prompt templates and parameters.



OGX Stack

The standardized server-side API loop for Llama and multi-model inference.



Jupyter

The interactive coding environment used to build our RAG pipelines today.

* Note: Advanced features like pipelines and model serving are available, but not our workshop focus today.

OpenShift AI Dashboard

 **Red Hat**
OpenShift AI

[Home](#)
[Projects](#)
[AI hub](#) ▾
Models
[Gen AI studio](#) ▾
Playground Tech Preview
AI asset endpoints Tech Preview
Prompts
[Develop & train](#) ▾
Pipelines ▸
Experiments (MLflow)
[Learning resources](#)
[Applications](#) ▸
[Settings](#) ▸

Feature flags are overridden in the current session. [Click here](#) to reset back to defaults.

 **Projects**



Start by creating your project

From workbenches to model servers, your project can be organized and customized to meet your needs.

[Create a project →](#)

[Go to Projects](#)

▾ **Get oriented with learning resources**



Creating a basic workbench

Quick start 10 minutes

This quick start walks you through creating a



OpenShift AI tutorial - Fraud detection example

by Red Hat

Tutorial 1 hour



Red Hat OpenShift AI

by Red Hat

Documentation



IBM watsonx.ai use case

by IBM

How-to 45 minutes

OpenShift AI: Workbenches

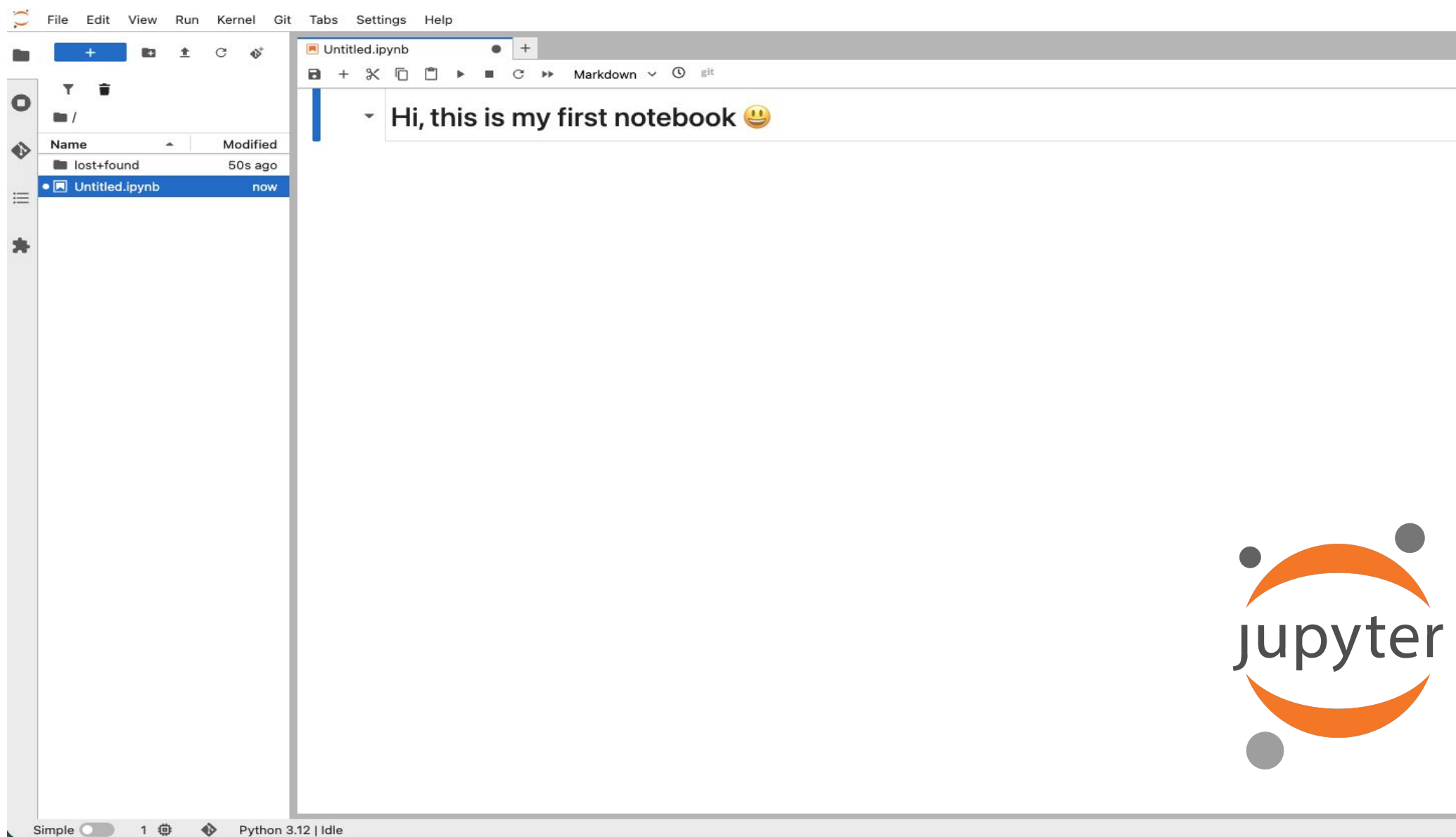
The screenshot displays the OpenShift AI console interface. The top navigation bar includes the Red Hat OpenShift AI logo, a notification bell, a grid icon, a help icon, a settings icon, a moon icon, and the user 'kubeadmin'. A blue banner at the top states: 'Feature flags are overridden in the current session. [Click here](#) to reset back to defaults.'

The left sidebar contains the following menu items: Home, Projects, AI hub, Models, Gen AI studio, Playground (marked 'Tech Preview'), AI asset endpoints (marked 'Tech Preview'), Prompts, Develop & train, Pipelines, Experiments (MLflow), Learning resources, Applications, and Settings.

The main content area shows the 'demo-app' project page. The 'Workbenches' tab is selected, displaying the 'Train models' section. This section includes two cards:

- Workbenches**: A card with an icon of a person at a desk. It describes a workbench as an isolated area for working with models in an IDE like Jupyter. It includes a 'Create a workbench' button.
- Pipelines**: A card with an icon of a pipeline. It states that to create or use pipelines, a pipeline server must first be configured. It includes a 'Configure pipeline server' link.

OpenShift AI: Jupyter Notebooks



The screenshot shows a Jupyter Notebook with four sections, each containing code to be executed in a cell. The notebook is titled 'redbank-demo-notebook.ipynb' and is running on Python 3.12. The left sidebar shows a file explorer with a table of files.

Name	Modified
lost+found	50s ago
redbank-demo-n...	now
Untitled.ipynb	3m ago

1. Install the llama stack client and other libraries

```
[ ]: %pip install llama_stack==0.3.0
      %pip install sounddevice
      %pip install kokoro
```

2. List available models

```
[ ]: from llama_stack_client import LlamaStackClient

      client = LlamaStackClient(base_url="http://localhost:8321")
      client.models.list()
```

3. Set the LLM and Embedding Model

```
[ ]: from llama_stack_client import LlamaStackClient

      client = LlamaStackClient(base_url="http://localhost:8321")

      models = client.models.list()

      # Select the LLM and first embedding models
      model_id = "ollama/qwen2.5:3b"
      embedding_model_id = (
          em := next(m for m in models if m.model_type == "embedding")
      ).identifier
      embedding_dimension = em.metadata["embedding_dimension"]

      print(model_id)
      print(embedding_model_id)
```

4. Ask LLM a question about Red Bank Financial

```
[ ]: from llama_stack_client import Agent, AgentEventLogger, LlamaStackClient

      agent = Agent(
          client,
          model=model_id,
          instructions="""
```

At the bottom of the interface, there is a status bar showing 'Simple' mode, 'Python 3.12 | Idle', and 'Mode: Command'. The bottom right corner indicates 'Ln 1, Col 1' and the file name 'redbank-demo-notebook.ipynb'.

OpenShift AI: GenAI Playground

The screenshot displays the OpenShift AI GenAI Playground interface. On the left is a sidebar with navigation links: Home, Projects, AI hub, Gen AI studio, Playground (highlighted with a 'Tech Preview' badge), AI asset endpoints (also with a 'Tech Preview' badge), Develop & train, Learning resources, Applications, and Settings. The main area is titled 'Playground' and shows a chat conversation. The chat starts with a system message: 'Hello, admin. Welcome to the model playground.' followed by a user prompt: 'What is the capitol of France.' and an AI response: 'The capital of France is Paris.' The right sidebar contains configuration options: 'Model details' (set to 'vllm-inference-1/RedHatAI/Llama-3.1-8B-Instruct'), 'System instructions' (a text box with a pre-defined assistant role), 'Temperature' (a slider set to 0.1), 'Streaming' (a toggle switch that is turned on), 'RAG' (a toggle switch that is turned off), and 'MCP servers' (a section indicating 'No MCP configuration found' with a message to contact the cluster administrator).

Red Hat OpenShift AI

Home Projects AI hub Gen AI studio Playground **Tech Preview** AI asset endpoints **Tech Preview** Develop & train Learning resources Applications Settings

Playground Project demo-ai View Code

Hello, admin
Welcome to the model playground.

Bot 12/1/2025, 6:17:46 PM
Send a message to test your configuration

admin 12/1/2025, 6:17:46 PM
What is the capitol of France.

Bot 12/1/2025, 6:17:46 PM
The capital of France is Paris.

Model details

Model
vllm-inference-1/RedHatAI/Llama-3.1-8B-Instruct

System instructions
You are a helpful AI assistant. You are designed to answer questions in a concise and professional manner.

Temperature ⓘ
0.1

Streaming

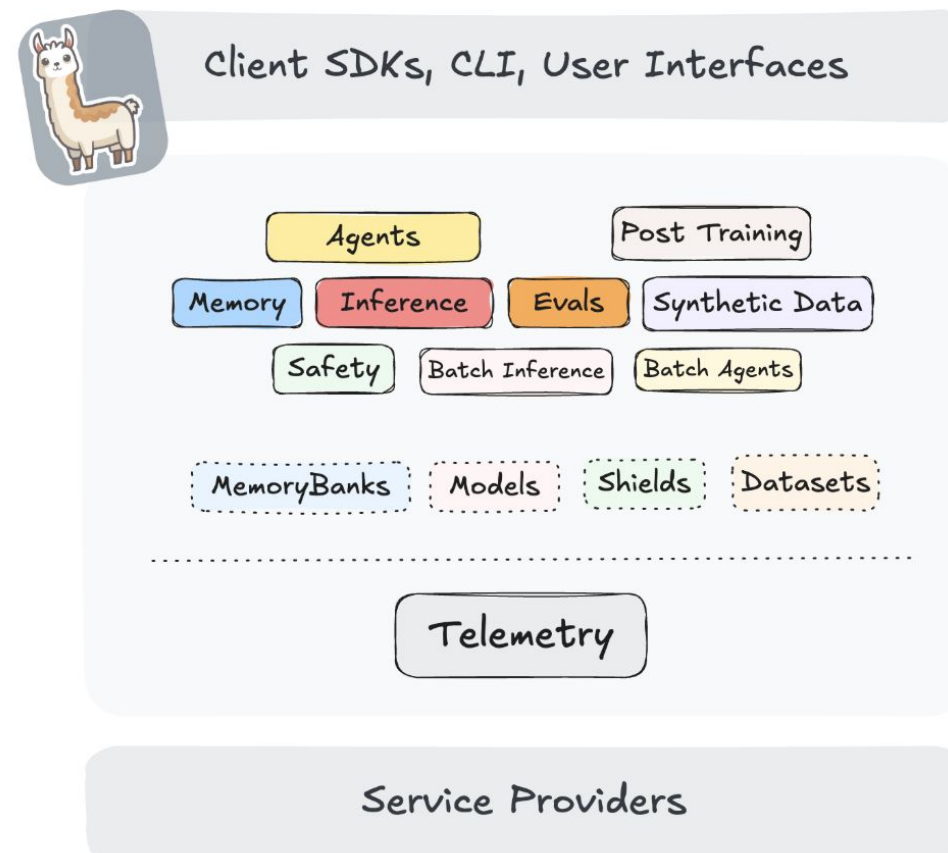
RAG

MCP servers

No MCP configuration found
This playground does not have an MCP configuration. Contact your cluster administrator to add MCP servers.

OpenShift AI: OGX Stack

- ▶ **VectorStore**: perform operations on vector DBs, such as adding documents, searching, and deleting documents
- ▶ **Tooling**: integrate with tools using MCP protocol
- ▶ **Safety**: apply safety policies (including Guardrails) to the output at a Systems (not only model) level
- ▶ **Inference** (Responses API): run inference with a LLM and multi-step agentic workflows with LLMs with tool usage, memory (RAG), etc.
- ▶ **Eval**: generate outputs (via Inference or Agents) and perform scoring
- ▶ **Telemetry**: collect telemetry data from the system



Let's talk about LLMs and RAG

LLMs = Large Language Models

How LLMs Learn and Specialize

Most people have already interacted with an AI model. Behind the scenes, models like **ChatGPT, Claude, Gemini, and DeepSeek** rely on unique **learned weights** acquired during training on vast datasets. This is why each model specializes in different tasks and behaves uniquely.

How LLMs Acquire New Knowledge:

- **Training & Fine-Tuning:** Directly modifying and updating the model's core weights using custom datasets.
- **Retrieval-Augmented Generation (RAG):** Dynamically injecting real-time, relevant context into the prompt without altering the weights.

EMERGING AI LLM ECOSYSTEM



ChatGPT (OpenAI)
Versatile Assistant, Chat & Code



Claude (Anthropic)
High-Fidelity, Ethical Reasoning



Gemini (Google)
Multimodal Power, Contextual Understanding



DeepSeek
Specialized, Open-Source & Coding

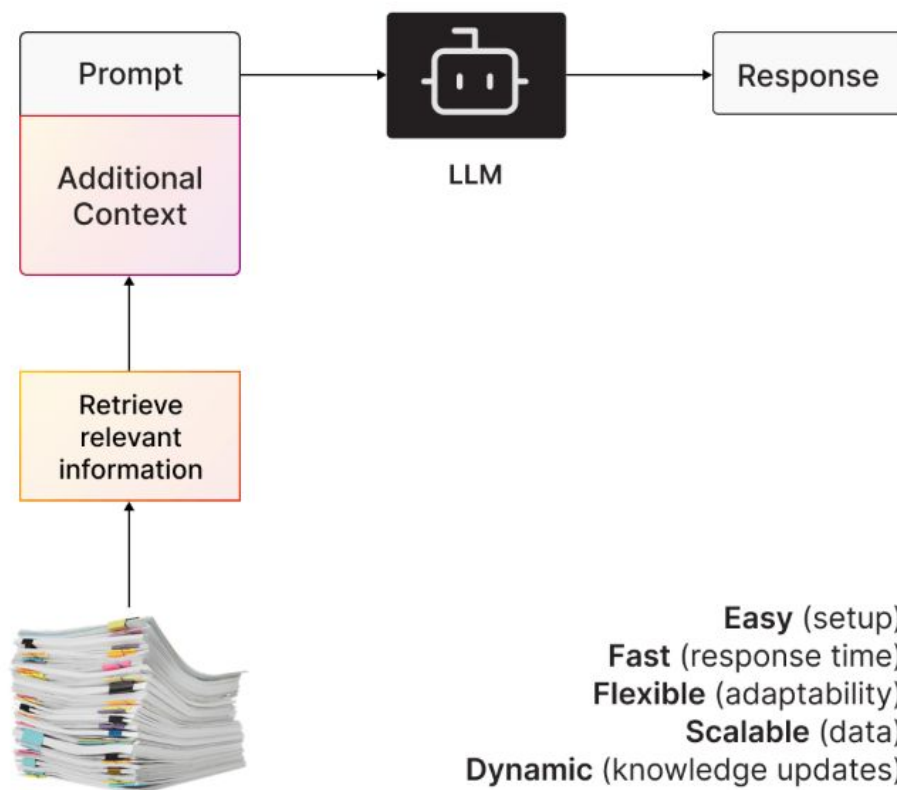
MODERN AI TECHNOLOGIES | 2024

PRESENTATION SLIDE - VISUAL OVERVIEW

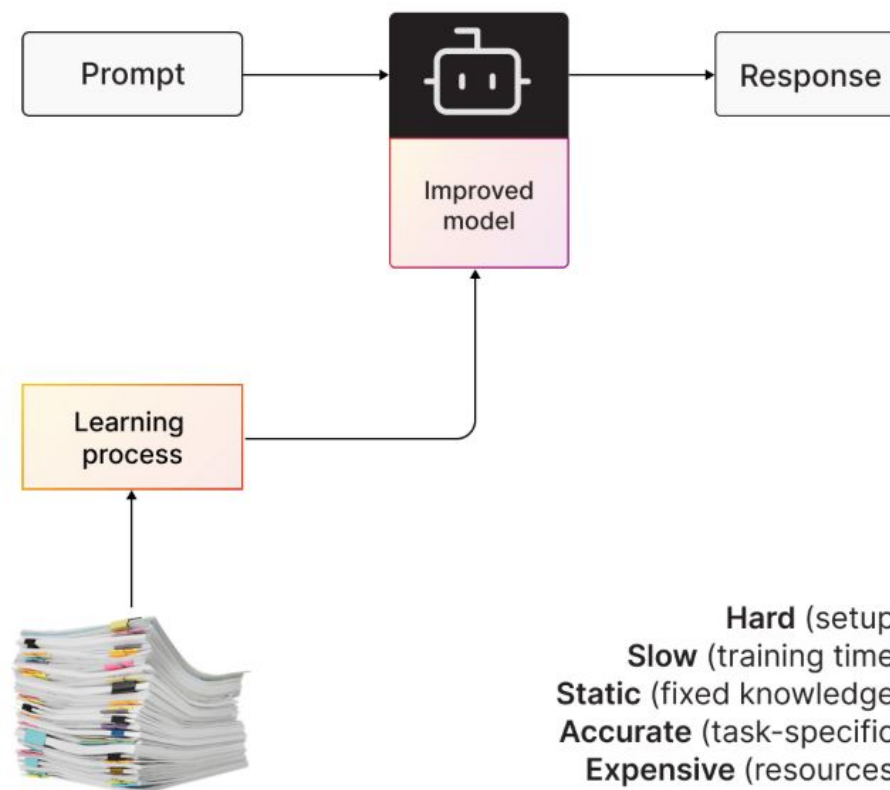
There are also Text-To-Speech, Speech-To-Text, and Embeddings models.

Why RAG instead of Fine Tuning?

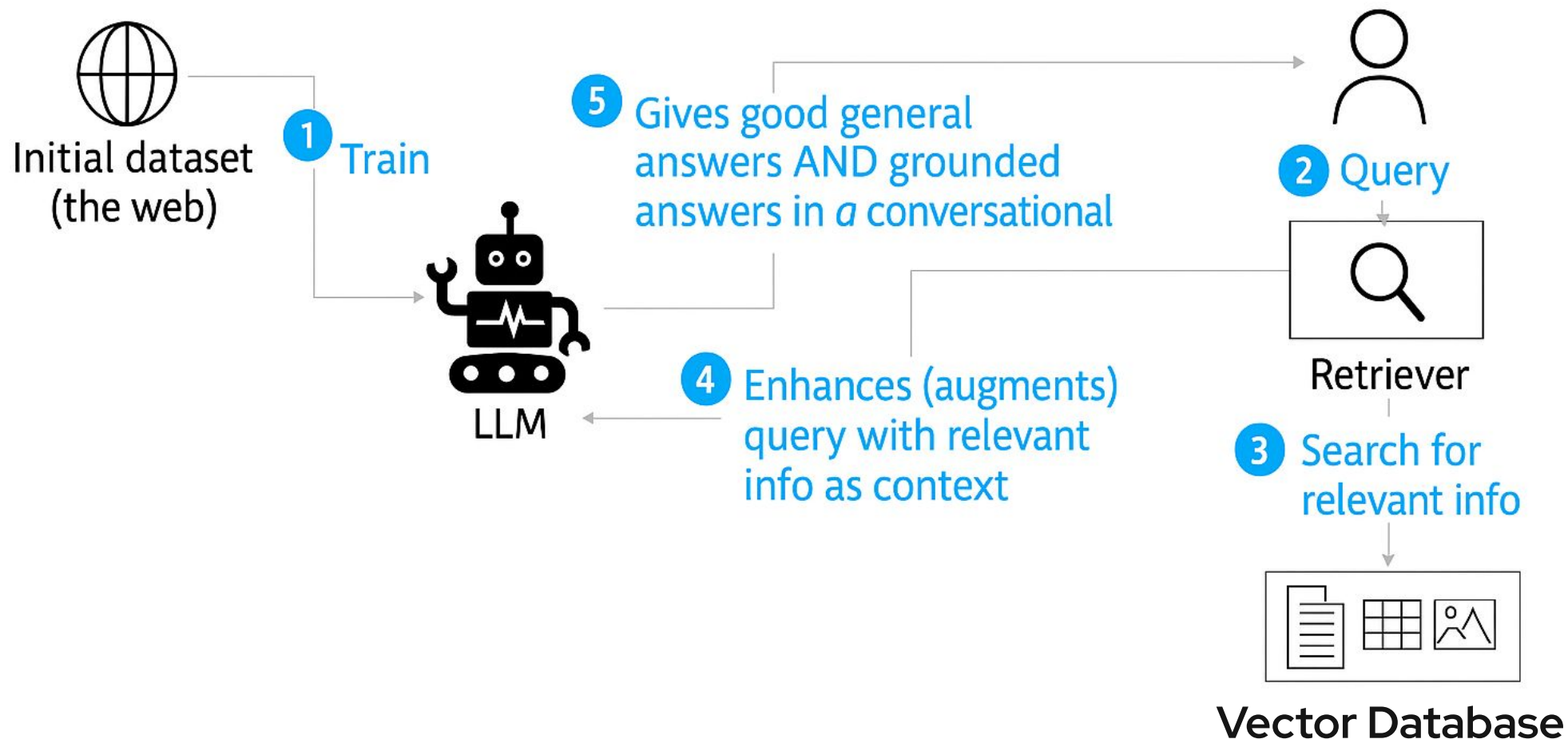
Retrieval-augmented generation (RAG)



Fine-tuning the Model



How does RAG work?



Concept: Text Chunking

Chunk Size:

The maximum length of text to include in a chunk.

Balanced chunk size preserves context and precision.

Typically measured in tokens.

Chunk Overlap Size:

Adds small amount of overlapping text between consecutive chunks.

Helps preserve continuity.

Text Chunking example

Retrieval-Augmented Generation, or RAG, bridges the gap between static language models and dynamic, knowledge-driven reasoning. Traditional LLMs

Upload .txt

Splitter: Character Splitter  

Chunk Size: 300

Chunk Overlap: 0

Total Characters: 834

Number of chunks: 3

Average chunk size: 278.0

Retrieval-Augmented Generation, or RAG, bridges the gap between static language models and dynamic, knowledge-driven reasoning. Traditional LLMs generate answers based on patterns learned during training, but their knowledge becomes stale the moment training ends. This can be overcome thanks to RAG. This makes responses more accurate, current, and context-aware without retraining the model itself. The process depends on embedding models that convert both user queries and documents into vector representations stored in a vector database. When a query is made, the RAG system retrieves the most semantically similar chunks, which are appended to the prompt before generation. This design decouples reasoning from stored knowledge, enabling smaller models to compete with much larger ones while dramatically lowering compute costs.

Text Chunking example

Retrieval-Augmented Generation, or RAG, bridges the gap between static language models and dynamic, knowledge-driven reasoning. Traditional LLMs

Upload .txt

Splitter: Character Splitter  

Chunk Size: 

Chunk Overlap: 

Total Characters: 874

Number of chunks: 3

Average chunk size: 291.3

Retrieval-Augmented Generation, or RAG, bridges the gap between static language models and dynamic, knowledge-driven reasoning. Traditional LLMs generate answers based on patterns learned during training, but their knowledge becomes stale the moment training ends. This can be overcome thanks to RAG. This makes responses more accurate, current, and context-aware without retraining the model itself. The process depends on embedding models that convert both user queries and documents into vector representations stored in a vector database. When a query is made, the RAG system retrieves the most semantically similar chunks, which are appended to the prompt before generation. This design decouples reasoning from stored knowledge, enabling smaller models to compete with much larger ones while dramatically lowering compute costs.

Concept: Embedding Models

Purpose:

Capture semantic meaning for accurate retrieval and insertion.

Convert text chunks to embeddings.

Benefits:

Enables semantic search.

Makes LLMs context-aware beyond their training data.

Limitations:

Dependant on the quality of the embedding model.

Embedding Models

Model	Nomic Embed	OpenAI Embeddings
Size	Smaller (768-dim)	Larger (1,536-dim)
Cost	Free, runs locally	Pay-per-token (API-based)
Accuracy	Strong semantic accuracy	Good, but not always top-performing
Latency	Very fast (no API call)	Higher (network latency)
Efficiency	Dense, compact vectors	Larger, sometimes redundant vectors
Best for	Local, cost-efficient RAG	Maximum semantic coverage

Vector Embeddings example



Text Chunks

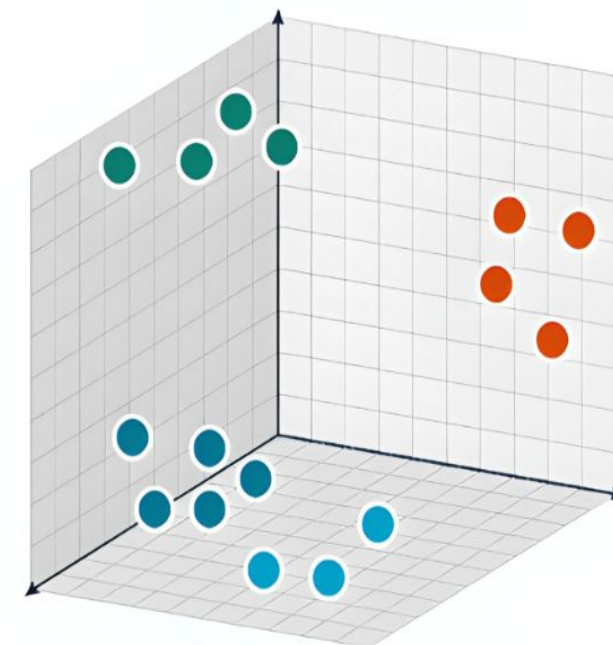


VECTOR EMBEDDING

[0.95, 0.49, 0.67, 0.75, 0.43, 0.00, 0.60, 0.13, 0.72, ...],
[0.73, 0.83, 0.80, 0.48, 0.55, 0.83, 0.86, 0.85, 0.06, ...],
[0.03, 0.71, 0.62, 0.07, 0.74, 0.30, 0.80, 0.08, 0.63, ...],
[0.49, 0.48, 0.38, 0.51, 0.45, 0.05, 0.10, 0.92, 0.29, ...],
[0.84, 0.38, 0.81, 0.00, 0.48, 0.61, 0.41, 0.85, 0.93, ...],
[0.24, 0.90, 0.69, 0.57, 0.43, 0.60, 0.37, 0.63, 0.28, ...],
[0.72, 0.33, 0.36, 0.98, 0.88, 0.05, 0.64, 0.91, 0.15, ...],
[0.89, 0.17, 0.44, 0.78, 0.33, 0.59, 0.29, 0.23, 0.22, ...],
[0.88, 0.75, 0.38, 0.77, 0.31, 0.98, 0.84, 0.79, 0.25, ...],
[0.11, 0.77, 0.30, 0.56, 0.62, 0.14, 0.89, 0.19, 0.62, ...],
[0.63, 0.41, 0.97, 0.73, 0.64, 0.41, 0.96, 0.36, 0.87, ...],
[0.51, 0.16, 0.26, 0.94, 0.53, 0.38, 0.39, 0.90, 0.55, ...],
[0.91, 0.99, 0.10, 0.87, 0.24, 0.91, 0.94, 0.36, 0.69, ...],
[0.89, 0.64, 0.14, 0.94, 0.51, 0.24, 0.61, 0.99, 0.11, ...],
[0.77, 0.65, 0.48, 0.94, 0.49, 1.00, 0.24, 0.10, 0.97, ...],
[0.58, 0.47, 0.01, 0.56, 0.59, 0.32, 0.79, 0.95, 0.40, ...]



VECTOR DATABASE



Vector Embeddings example



Give me information
about a Poodle ●

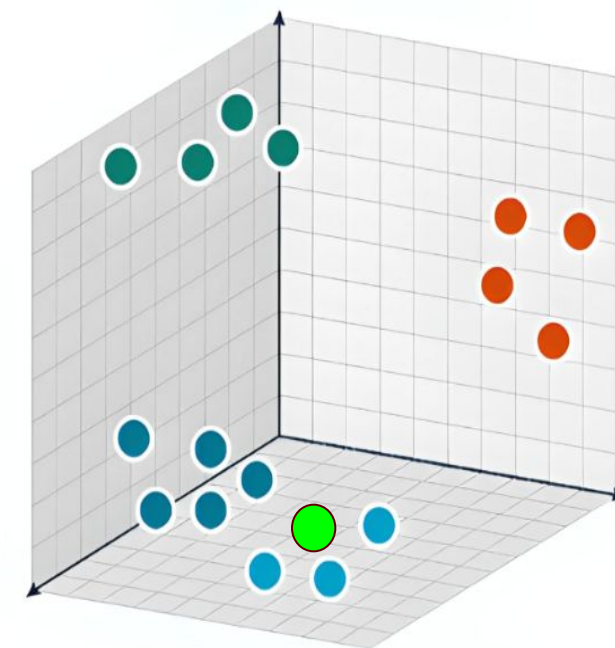


VECTOR EMBEDDING

[0.24, 0.90, 0.69, 0.57, 0.43, ...]



VECTOR DATABASE



Approximate Nearest Neighbor (ANN)



Vector Databases - Search Modes

Query: I would like information about a **pet**?

Search Modes:

Keyword search: exact terms present.

`(Pet == Pet)`

Vector search: based on semantic similarity in embedding space.

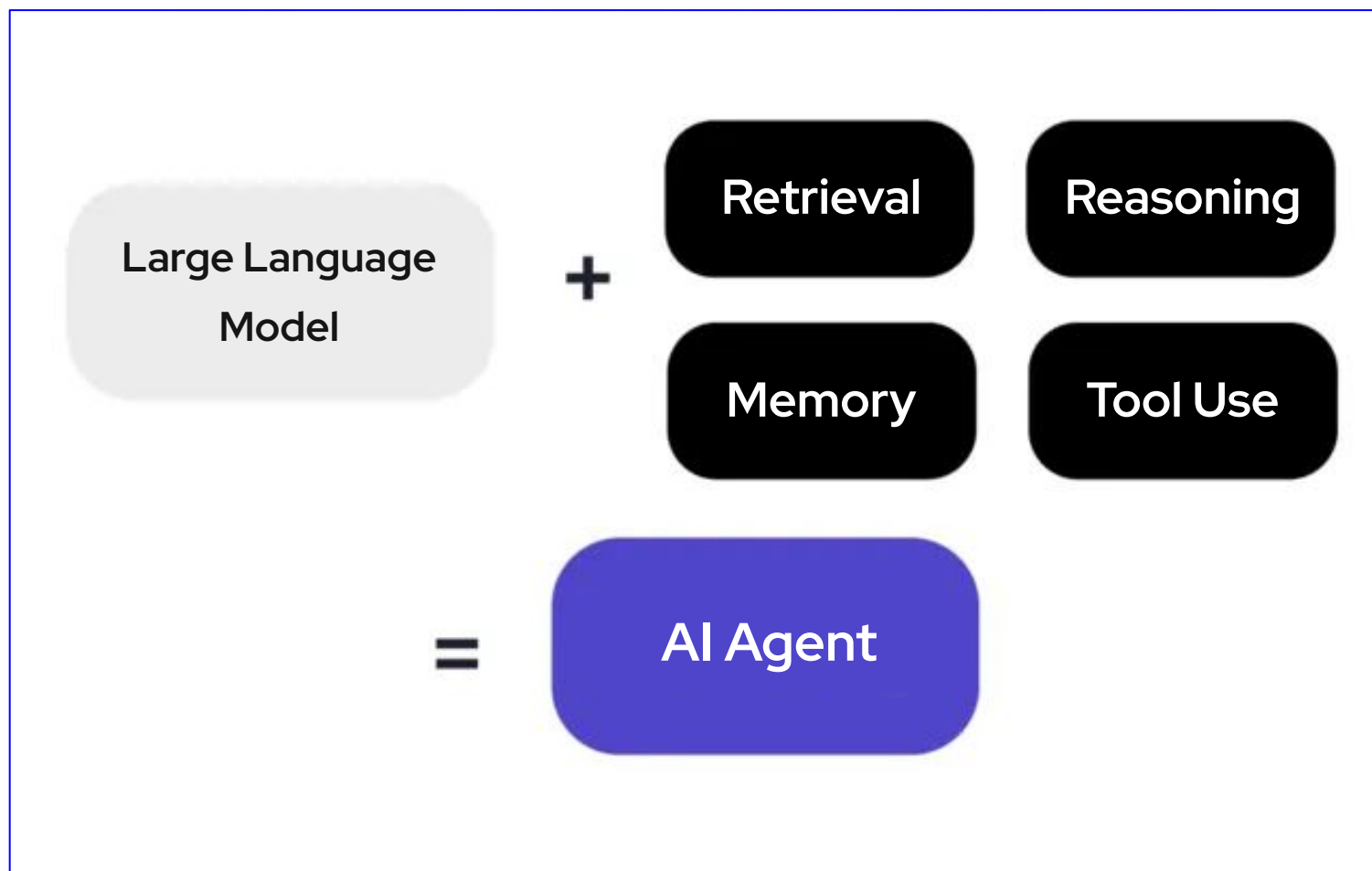
`(Pet == Pet) or (Pet == Dog) or (Pet == Cat)`

Hybrid search: combines both using a weighting factor (e.g. alpha).

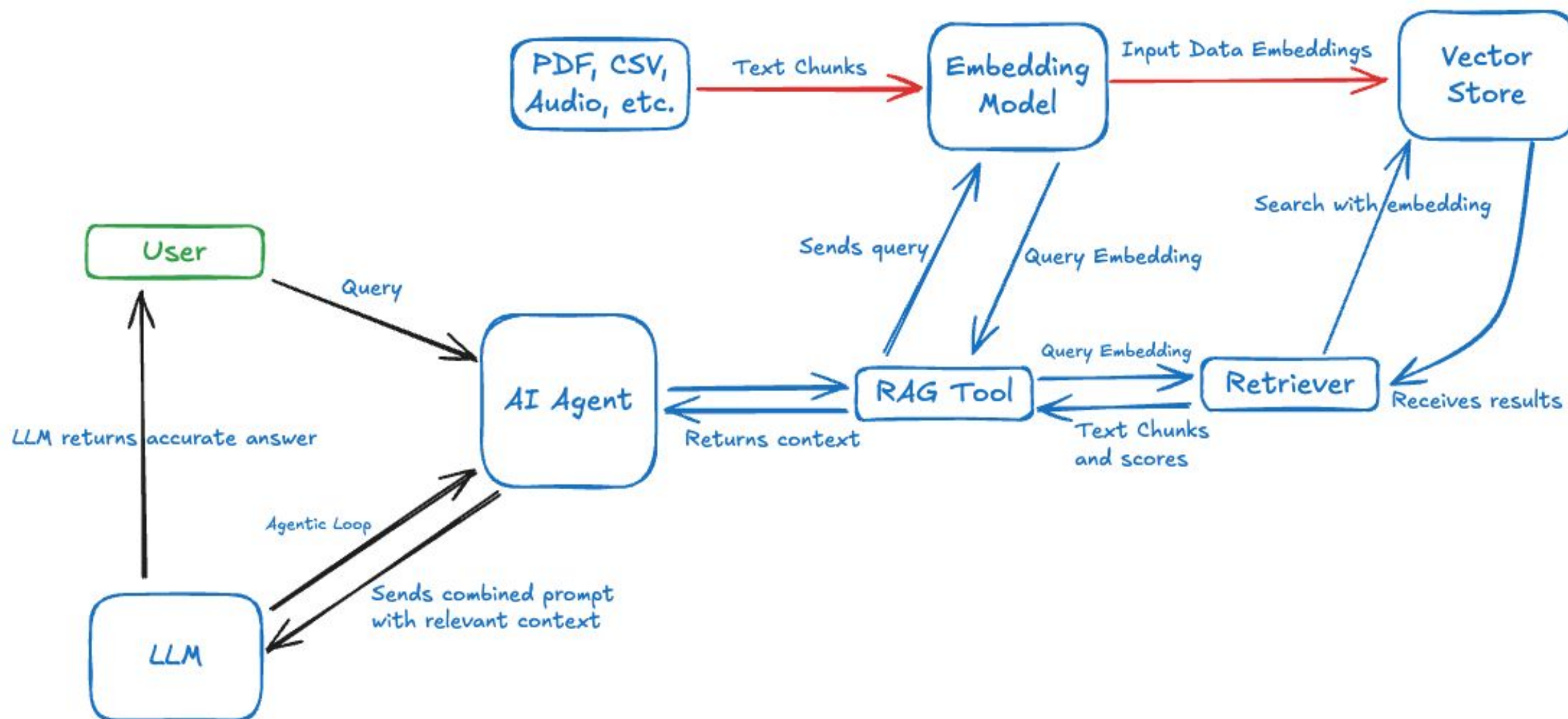
$$\text{hybrid_score} = \alpha \times \text{lexical_score} + (1-\alpha) \times \text{vector_score}$$

Concept: AI Agents

Query → Agent decision → Tool(s) → Response.



Joining them together



RAG Workshop with OpenShift AI

Red Bank Financial



Use Case – Red Bank Financial

Our use case is a customer service scenario for Red Bank Financial

- **Pain points:**

- Long queue times.
- Callers must listen to every menu option before selecting the right one.
- Limited 24/7 availability.
- Long resolution times.

- 
- A red-outlined icon of a document with a folded top-right corner, containing a bulleted list.
- History
 - Address
 - Phone number
 - Email



Use Case – Red Bank Financial

Our use case is a customer service scenario for Red Bank Financial

If we ask an LLM about Red Bank Financial:

It will have no knowledge.

What if...

We provide documents from Red Bank Financial with company information.

With RAG:

The model can now answer accurately to requests.
Example: "What is Red Bank Financial?" → reliable answer.

- 
- History
 - Address
 - Phone number
 - Email



Solution Architecture but with MaaS and external models

